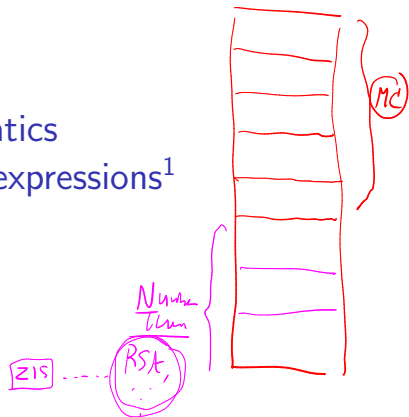
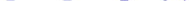


01204211 Discrete Mathematics
Lecture 7a: Languages and regular expressions¹

Jittat Fakcharoenphol

August 10, 2026



¹Based on lecture notes of *Models of Computation* course by Jeff Erickson. 

What is computation?

→ ကိန်းဂဏန်း အကဲဖြတ်မှု စနစ်

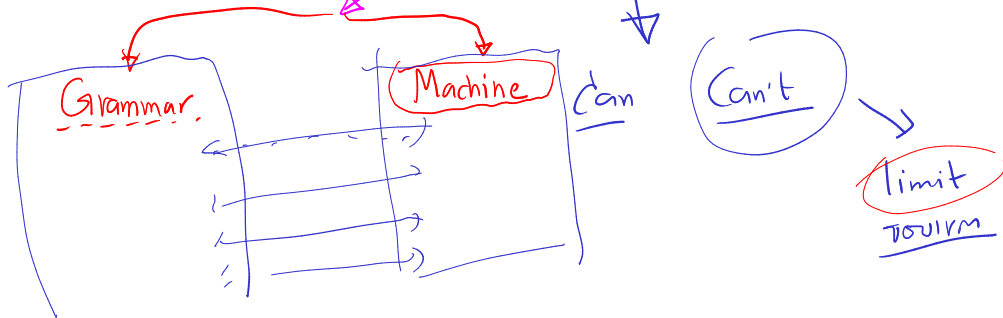
- ပြဿနာ instructions
 - ကိန်းဂဏန်း
- Instructions

Set of strings Σ

Problem

Definition

Solving problem



Models of computations

→ definitions

→ abilities

→ limits

Languages = specifications

formal
languages

Set of strings

$$\Sigma = \{0, 1\}$$

$$\text{EVEN} = \{0, 10, 110, 01010, \dots\} = \{w \mid w \text{ is a binary string that is even}\}$$

$$\text{ODD} = \{1, 11, 01, 111, \dots\} = \{w \mid w \text{ is a binary string that is odd}\}$$

$$\text{PRIME} = \{10, 11, 101, 111, \dots\} = \{w \mid w \text{ is a binary string that is prime}\}$$

but language L ,
but input w ,
ברור $w \in L$?

"natural
languages"

Σ = unicode character

PYTHON = $\{p \mid p \text{ is a string that is a Python program}\}$

Formal definition: strings

$$\Sigma^* = \{\epsilon, 0, 1, 00, 01, 10, 11, 000, 001, \dots\}$$

Intuitively, a string is a *finite* sequence of symbols. However, to be able to formally prove properties of strings we need a precise definition.

Let a **finite** set Σ be the **alphabet**. (E.g., for bit strings, $\Sigma = \{0, 1\}$; for digits, $\Sigma = \{0, 1, \dots, 9\}$; for English string $\Sigma = \{a, b, \dots, z\}$.)

The following is a recursive definition of strings.

Recursive definition of strings

A **string** w over alphabet Σ is either

- ▶ the empty string ϵ , or
- ▶ $a \cdot x$ where $a \in \Sigma$ and x is a string.

The set of all strings over alphabet Σ is denoted by Σ^* .

Review: more recursive definitions

Lengths

For a string w , let $|w|$ be the length of w defined as

$$|w| = \begin{cases} 0 & \text{when } w = \varepsilon \\ 1 + |x| & \text{when } w = a \cdot x \end{cases}$$

Concatenation

For strings w and z , the concatenation $w \cdot z$ is defined recursively as

$$w \cdot z = \begin{cases} z & \text{when } w = \varepsilon \\ a \cdot (x \cdot z) & \text{when } w = a \cdot x \end{cases}$$

Review: proving facts about strings

Lemma 1

For strings w and x , $|w \cdot x| = |w| + |x|$.

$$w \cdot x = \begin{cases} x & \text{when } w = \varepsilon \\ a \cdot (y \cdot x) & \text{when } w = a \cdot y \end{cases}$$

$$|w| = \begin{cases} 0 & \text{when } w = \varepsilon \\ 1 + |x| & \text{when } w = a \cdot x \end{cases}$$

Proof. ទំនាក់ទំនង by induction លើ w

I.H. ដឹងថាបំពេញ string y អំពី $|y| < |w|$, $|y \cdot x| = |y| + |x|$.

Case $w = \varepsilon$

- ① $w = \varepsilon$ គឺជា assumption
- ② $w \cdot x = x$ គឺជាជំនួយការ
- ③ $|w \cdot x| = |x|$ គឺជា 2
- ④ $|w| = 0$ គឺជា ជំនួយការ 1
- ⑤ $|w \cdot x| = |x| = 0 + |x| = |w| + |x|$ គឺជាជំនួយការ



Review: proving facts about strings

Lemma 1

For strings w and x , $|w \cdot x| = |w| + |x|$.

Proof.

I.H. สำหรับทุก string y ที่ $|y| < |w|$, $|y \cdot x| = |y| + |x|$.

Case $w = a \cdot y$ เมื่อ y เป็น string ว่าง.

① $|w| = |a \cdot y| = 1 + |y|$ จกนิยาม 1

② $w \cdot x = a \cdot (y \cdot x)$ จกนิยาม 2

③ $|y \cdot x| = |y| + |x|$ I.H.

④ $|w \cdot x| \overset{\text{จก ②}}{=} |a \cdot (y \cdot x)| \overset{\text{จกนิยาม 1}}{=} 1 + |y \cdot x| \overset{\text{จก ③}}{=} 1 + |y| + |x| \overset{\text{จก ①}}{=} |w| + |x|$ จบการพิสูจน์

► เมื่อตรวจสอบว่าพจน์สำหรับ w เป็นจริง, Lemma นี้จึงเป็นจริงโดย induction



Formal languages

A **formal language** is a set of strings over some finite alphabet Σ .

Examples: $\Sigma = \{0, 1\}$

0
01
0001
000111

$$A = \{ w \in \Sigma^* \mid w \text{ มี 2 น. 0 และ 1 เท่ากัน} \}$$

$$B = \{ w \in \Sigma^* \mid w \text{ มี 1 ก่อนแล้วจึงมี 0 แล้วจึงมี 1 แล้วจึงมี 0} \}$$

010x

01110111x

$\emptyset$

$$\{011\}$$

Σ
01
10
0011
0101
0110
1010
1100
...

Careful...

These are different languages: $\underline{\emptyset}, \underline{\{\varepsilon\}}$
And ε is not a language.



How to describe languages?

① បំពេញការងារ / ជួយ set $\{w \in \Sigma^* \mid w \dots\}$

ex: $\{w \in \Sigma^* \mid w \text{ គឺជា string ដែលមានលក្ខណៈស្មើគ្នា } w = 1^n\}$

→ more, specification

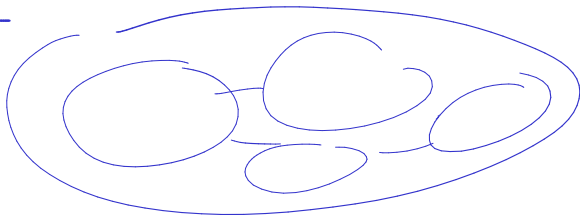
② language ជួយ set - ជួយ ធានាភាពជាក់លាក់

$$A = \{00, 011, 1011, 11110\}$$

③ ជួយស្រាវជ្រាវ string ក្នុង language

④ _____

Composition -



(Language) $\rightarrow$ set of strings

25th language
in larger set

L_1, L_2

① L_1^c

② $L_1 \cup L_2$

③ $L_1 \cap L_2$

④ $L_1 - L_2$

Combining languages

If A and B are languages over alphabet Σ .

- ▶ Basic set operations: $A \cup B$, $A \cap B$, $\bar{A} = \Sigma^* \setminus A$.
- ▶ Concatenation: $A \cdot B$.

$$A \cdot B = \{ a \cdot b \mid a \in A, b \in B \}$$

$$A = \{0, 01, 10, 11\}$$

$$B = \{0, 11, 111\}$$

$$A \cdot B = \{00, 010, 100, 110, 011, 1111, 1011, 11111, \dots\}$$

- ▶ Kleene closure or Kleene star: A^* ← concatenation of strings in A

Case 1: $\epsilon \in A^*$

Case 2: ถ้ามี $x \in A$ แล้ว $y \in A^*$, $x \cdot y \in A^*$

นั่นคือถ้า A^* มีสมาชิก x และ y แล้ว $x \cdot y$ ก็เป็นสมาชิกใน A^*

Also $A^+ = A \cdot A^*$

Examples

Regular languages

Definition: regular languages

A language L is **regular** if and only if it satisfies one of the following conditions:

- ✓ ▶ L is empty; $\emptyset$
- ✓ ▶ L contains one string (can be the empty string ε);
- ▶ L is a union of two regular languages;
- ▶ L is the concatenation of two regular languages; or
- ▶ L is the Kleene closure of a regular language.

Examples

$$\Sigma = \{0, 1\}$$

$$\{0, 11\}$$

$$\underbrace{\phi}_{\checkmark}$$

$$\underbrace{\{0\}}_{\checkmark}$$

$$\underbrace{\{11\}}_{\checkmark}$$

$$\underbrace{(\phi \cup \{0\})}_{\checkmark} \bullet \underbrace{(\{11\})}_{\checkmark}$$

$$\{0\} \cup \{11\}$$

$$(\{0\} \cup \{11\})^*$$

$$\{0\}^* \bullet \{11\}^*$$

$$(\{0\} \cup \{11\})^* = \Sigma^*$$

Examples

$\{11\}$

$\{0\}$
✓

$\{\epsilon\}$
✓

1 $\emptyset$
2 $\cup$
3 $\cup$
4 $\cdot$
5 $*$

$$\{11\} \cup \{0\} = \{11, 0\}$$

regular language.

$$(\{11\} \cup \{0\})^* \cdot \{110\}$$

$\{11, 0\}^*$

regular expression

$$(11 + 0)^* 110$$

Regular expressions

Let $\Sigma = \{0, 1\}$. Consider

$$\underbrace{\left(\underbrace{\{01\}} \cup \underbrace{\left(\{1\} \cdot \underbrace{(\{0\} \cup \{10\})} \right) \right)} \cup \underbrace{\left(\{00\} \cdot \underbrace{(\{1\})^*} \right)} \cdot \underbrace{\left(\underbrace{(\{0\} \cdot \{0\})} \cdot \{1\} \right)}$$

$\left((01 + 1(0 + 10)) + 001^* \right) 001$

Regular expressions

Regular language

$$((\{01\} \cup (\{1\} \cdot (\{0\} \cup \{10\}))) \cup (\{00\} \cdot (\{1\})^*)) \cdot ((\{0\} \cdot \{0\}) \cdot \{1\})$$

is represented as

$$(01 + 1(0 + 10) + 00(1)^*)001$$

Regular expressions

- ▶ omit braces around one-string sets
- ▶ use $+$ instead of $\cup$
- ▶ omit $\cdot$
- ▶ follow the precedence: Kleene star operator $*$, $\cdot$ (implicitly), and $+$.

Remark: $+$ and $\cdot$ are associative, i.e., $(A + B) + C = A + (B + C)$ and $(A \cdot B) \cdot C = A \cdot (B \cdot C)$.

Regular expressions: examples 1

$$(0+1)^*1$$

set no
strings that end with 00 in strings

$$(0+1)^*00(0+1)^*$$

$$(0+1)^*0000000000$$

$$\rightarrow \boxed{(0+1)^*010(0+1)^*}$$
$$(0+1)^*0(0+1)^*1(0+1)^*0(0+1)^*$$
$$\underline{010} \underline{10} \underline{10} \underline{10}$$

Regular expressions: examples 2

All strings over $\{0, 1\}$ except 010.

$$\varepsilon + \underline{1}(0+1)^* + \underline{00}(0+1)^* + \underline{01}(\varepsilon + 1(0+1)^*)$$

(d)

$$0^* +$$

$$\uparrow \quad +0$$

$$\boxed{\Sigma^* - \{010\}}$$

$$\boxed{0^*1^* + (0^*\underline{11}1^*0^*)^*1^*}$$

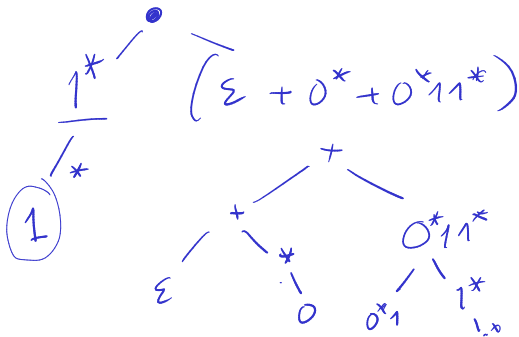
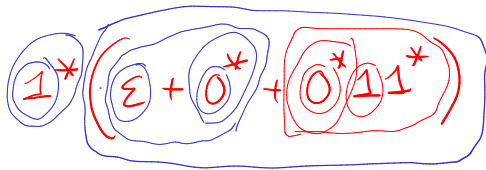
↑

$$\boxed{\Sigma^* \cancel{-} (0+1)^*010(0+1)^*}$$

(e)

$$1^* \left(\varepsilon + 0^* + 0^*11^* \right)$$

Subexpressions



Regex is everywhere

Proofs about regular expressions - structural induction

for string w , (w^R) (reversal of w)

$$(hello)^R = olleh$$

$$(a \cdot x)^R$$

$$(h \cdot (ello))^R = olle \cdot h$$

$$\begin{aligned} (math)^R &= (ath)^R \cdot m \\ &= ((th)^R \cdot a) \cdot m \end{aligned}$$

ex: $L = \{hello, good, bad, cpe\}$

$L^R = \{olleh, doog, dab, epc\}$

w is a string

- $w = \epsilon$

- $w = a \cdot x$

also $a \in \Sigma$ is a char
x is a string

Definition

$$w^R = \begin{cases} \epsilon & \text{if } w = \epsilon \\ x^R \cdot a & \text{if } w = a \cdot x \end{cases}$$

also $a \in \Sigma$

also: x is a string

Definition

if L is a language

$$L^R = \{w^R \mid w \in L\}$$

Proofs about regular expressions - structural induction

the reversal of a regular lang is regular.

▶ An L is regular, L^R is also regular

ex: $L = \emptyset$ $L^R = \emptyset$ ist regular ✓

$L = \{11001\}$ $L^R = \{10011\}$ is regular?

$$L = \{\underline{1101}\} \cup \{\underline{10}\} \quad L^R = \{\underline{1011}\} \cup \{\underline{01}\}$$
$$\underline{L} = A \cup B$$
$$L^R = A^R U B^R$$

Proofs about regular expressions - structural induction

Proof: Show that regular language R for any.

• Case $R = \emptyset$

• Case $R = \{w\}$ for w the string

• Case $R = A \cup B$ for A & B the regular lang

• Case $R = A \cdot B$ for A & B reg lang

• Case $R = A^*$ for A the reg language



induction
hypothesis ?

Proofs about regular expressions - structural induction

Proof: Given regular language L and R is a regular expression if $L = L(R)$

[IH.] Assuming regular expression R' s.t. $|R'| < |R|$

• Case $R = \emptyset$

• Case $R = w$ where w is a string

• Case $R = A + B$ where A & B are regular expressions

• Case $R = A \cdot B$ where A & B are reg expressions

• Case $R = A^*$ where A is a reg expression

or L

xyz

assuming

subexpression (R')

and $R \dots$

Lemma 2

Every regular expression that does not use the symbol $\emptyset$ represents a non-empty language.

Proof. *Assume* regular expression R *does not use* $\emptyset$

I.H. *Assume* *any* subexpression R' *does not use* $\emptyset$, R' *represents* non-empty language.

Case 1: $R = \emptyset$ *is not possible*

Case 2: $R = w$ *is a string*

Language *represented by* R *is* string w , *is not* empty

Case 3: $R = A + B$ *is* A *or* B *is* reg expression

① A *is* sub expression R , *in I.H.*, A *represents* non empty language

② *Assume* R *represents* language *as* A *or* B *is* non empty

$\Rightarrow R$ *is* non empty

Lemma 2

Every regular expression that does not use the symbol $\emptyset$ represents a non-empty language.

$L(R)$
 $L(A)$
 $L(B)$

Proof.

Case 4: $R = A \cdot B$ and $L(A) \neq \emptyset$, $L(B) \neq \emptyset$
ดังนั้น R is empty

Case 5: $R = A^*$. Since $\epsilon \in A^*$ ดังนั้น R is non-empty

เนื่องจาก ปรกติแล้วถ้าไม่มี $\emptyset$ ในนิพจน์ 5 นั้น,
ปรกติแล้วมันจะไม่เป็น $\emptyset$.

Lemma 2

Every regular expression that does not use the symbol $\emptyset$ represents a non-empty language.

Proof.

Let R be a regular expression that does not use the symbol $\emptyset$. We prove by (structural) induction that R represents a non-empty language.

Lemma 2

Every regular expression that does not use the symbol $\emptyset$ represents a non-empty language.

Proof.

Let R be a regular expression that does not use the symbol $\emptyset$. We prove by (structural) induction that R represents a non-empty language.

Induction hypothesis: Every subexpression of R that does not use the symbol $\emptyset$ represents a non-empty language.

Lemma 2

Every regular expression that does not use the symbol $\emptyset$ represents a non-empty language.

Proof.

Let R be a regular expression that does not use the symbol $\emptyset$. We prove by (structural) induction that R represents a non-empty language.

Induction hypothesis: Every subexpression of R that does not use the symbol $\emptyset$ represents a non-empty language.

Case 1: $R = \emptyset$.

Lemma 2

Every regular expression that does not use the symbol $\emptyset$ represents a non-empty language.

Proof.

Let R be a regular expression that does not use the symbol $\emptyset$. We prove by (structural) induction that R represents a non-empty language.

Induction hypothesis: Every subexpression of R that does not use the symbol $\emptyset$ represents a non-empty language.

Case 1: $R = \emptyset$. 

Case 2: R is a single string. 

Lemma 2

Every regular expression that does not use the symbol $\emptyset$ represents a non-empty language.

Proof.

Let R be a regular expression that does not use the symbol $\emptyset$. We prove by (structural) induction that R represents a non-empty language.

Induction hypothesis: Every subexpression of R that does not use the symbol $\emptyset$ represents a non-empty language.

Case 1: $R = \emptyset$.

Case 2: R is a single string.

Proof. (cont.2/4)

Case 3: $R = S + T$ for some regular expressions S and T .

① If S is the sub expression in R then R is not $\emptyset$
if S is not $\emptyset$

② in I.H., $L(S) \neq \emptyset$

③ $L(R) = L(S+T) = L(S) \cup L(T) \neq \emptyset$. $\therefore$ $L(R) \neq \emptyset$

Proof. (cont.3/4)

Case 4: $R = (S \cdot T)$ for some regular expressions S and T .

- ① S is the sub expression in R and T is the sub expression in R
- ② in I.H., $L(S) \neq \emptyset$
- ③ in I.H., $L(T) \neq \emptyset$
- ④ in I.H., $L(S \cdot T) = L(S) \cdot L(T)$
- ⑤ in I.H., $L(S) \neq \emptyset$, $\exists$ string $w \in L(S)$
- ⑥ in I.H., $L(T) \neq \emptyset$, $\exists$ string $z \in L(T)$
- ⑦ in I.H., $w \cdot z \in L(S \cdot T) = L(R)$
- ⑧ $L(R) \neq \emptyset$

Proof. (cont.4/4)

Case 5: $R = S^*$ for some regular expression S .

$$\text{from } L(S^*) \ni \varepsilon, \quad L(S^*) = L(R) \neq \emptyset$$

Proof. (cont.4/4)

Case 5: $R = S^*$ for some regular expression S .

In every case, the language $L(R)$ is non-empty. ✓

Lemma 3

Every non-empty regular language is represented by a regular expression that does not use the symbol $\emptyset$.

Lemma 3

Every non-empty regular language is represented by a regular expression that does not use the symbol $\emptyset$.

Let R be a regular expression.

Lemma 3

Every non-empty regular language is represented by a regular expression that does not use the symbol $\emptyset$.

Let R be a regular expression. We prove that if $L(R) \neq \emptyset$, then there exists a regular expression R' such that $L(R) = L(R')$ and R' does not contain $\emptyset$.

Lemma 3

Every non-empty regular language is represented by a regular expression that does not use the symbol $\emptyset$.

Let R be a regular expression. We prove that if $L(R) \neq \emptyset$, then there exists a regular expression R' such that $L(R) = L(R')$ and R' does not contain $\emptyset$.

We prove by induction. What should the induction hypothesis be?

IH: In every subexpression S of R , if $L(S) \neq \emptyset$,
then there exists a regular expression S' such that $L(S') = L(S)$
and S' does not contain $\emptyset$

ex: $R = \emptyset + 111$
 $R' = 111$

I.H.: For every subexpression S of R , if $L(S) \neq \emptyset$, there exists an $\emptyset$ -free regular expression S' such that $L(S) = L(S')$.

Case: $R = S + T$ and $L(R) \neq \emptyset$ and also $L(S)$ and $L(T)$ is empty or not

Case 1: $L(S) \neq \emptyset, L(T) \neq \emptyset$

an I.H., by S and T and $L(S') = L(S), L(T') = L(T)$

and: S' and T' $\emptyset$ -free.

and for $R' = S' + T'$

① R' is $\emptyset$ -free

$$\begin{aligned} \text{② } L(R') &= L(S') \cup L(T') \\ &= L(S) \cup L(T) = L(R) \end{aligned}$$

on now.

I.H.: For every subexpression S of R , if $L(S) \neq \emptyset$, there exists an $\emptyset$ -free regular expression S' such that $L(S) = L(S')$.

Case 2: $L(S) \cap L(T) = \emptyset$, assuming $L(T) = \emptyset$
then $L(S) \neq \emptyset$ bbn: $L(R) = L(S) \cup L(T) = L(S)$.

An I.H. of reg ex S' of $\emptyset$ -free bbn: $L(S') = L(S)$

for $R' = S'$ assuming R' is $\emptyset$ -free

bbn: $L(R') = L(S') = L(S) = L(R)$

minim.

I.H.: For every subexpression S of R , if $L(S) \neq \emptyset$, there exists an $\emptyset$ -free regular expression S' such that $L(S) = L(S')$.

What are the cases that we have to consider?

▷ $R = \emptyset$

▷ $R = \omega$

▷ $R = S + T$ or $S \& T$ or ---

▷ $R = S \circ T$ —

▷ $R = S^*$ —

I.H.: For every subexpression S of R , if $L(S) \neq \emptyset$, there exists an $\emptyset$ -free regular expression S' such that $L(S) = L(S')$.

What are the cases that we have to consider?

- ▶ $R = \emptyset$
- ▶ R is a single string.
- ▶ $R = S + T$ for some regular expressions S and T . *
- ▶ $R = S \cdot T$ for some regular expressions S and T .
- ▶ $R = S^*$ for some regular expression S .

(E-ex1-6) For string w , the reversal w^R is defined recursively as follows:

$$w^R = \begin{cases} \varepsilon & \text{if } w = \varepsilon \\ x^R \cdot a & \text{if } w = ax \text{ for some symbol } a \text{ and some string } x \end{cases}$$

For a language L , the reversal of L is defined as

$$L^R = \{w^R \mid w \in L\}.$$

You may assume the following facts.

- ▶ $L^* \cdot L^* = L^*$ for every language L .
- ▶ $(w^R)^R = w$ for every string w .
- ▶ $(x \cdot y)^R = y^R \cdot x^R$ for all strings x and y .

Prove that $(L^R)^* \subseteq (L^*)^R$.